

Healthcare Intelligence Assistant

*A Retrieval-Augmented Generation Based
Healthcare Decision Support System*

Shambhavi Srivastava Yashash Yallapragada

Department of Data Science & Artificial Intelligence
IIT Madras Zanzibar Campus

Z2004 - Database Management Systems

Abstract

The *Healthcare Intelligence Assistant* is a Retrieval-Augmented Generation (RAG) system designed to provide evidence-based healthcare decision support. The system integrates Microsoft SQL Server for structured relational data management with PostgreSQL (Supabase pgvector) for semantic vector search. A corpus sourced from WHO, CDC, Mayo Clinic, and NIH is stored as over 1,183 semantic text chunks, embedded using the `all-MiniLM-L6-v2` model, and retrieved via cosine similarity search. The system is surfaced through a Streamlit web interface with additional features including symptom severity assessment and hospital recommendations via the OpenStreetMap Nominatim API. Performance was optimized using covering non-clustered indexes, achieving a 91.9% reduction in logical reads and a 60% improvement in query execution time. This project demonstrates the practical integration of relational databases, vector retrieval, and large language model techniques in a healthcare decision support context.

Contents

1	Introduction and Problem Statement	2
2	System Overview	2
3	Database Design	3
3.1	Schema Overview	3
3.2	ER Relationships and Cardinalities	3
3.3	Database Normalisation (3NF)	3
4	Dataset Construction	4
4.1	Data Sources and Topic Coverage	4
4.2	Chunk and Embedding Generation	4
5	Hybrid Database Architecture and Retrieval Pipeline	4
5.1	Architecture Rationale	4
5.2	End-to-End Retrieval Workflow	4
6	Application, Severity Classification, and Evaluation	5
6.1	Interactive Application	5
6.2	Severity Classification	5
6.3	Evaluation Framework	5
7	Database Performance Optimization	6
7.1	Index Design and Justification	6
7.2	Query 1 — Primary RAG Retrieval Join	7
7.3	Query 2 — Document Lookup by Medical Speciality	8
8	Results, Limitations, and Future Work	8
8.1	Key Outcomes	8
8.2	Limitations	9
8.3	Future Work	9
9	Conclusion	9
	References	10

1. Introduction and Problem Statement

The proliferation of digital healthcare information creates both an opportunity and a challenge: vast repositories of evidence-based medical knowledge are available online, yet end-users often lack the means to query this knowledge efficiently and receive grounded, reliable responses. Traditional keyword-based search returns document lists rather than synthesised answers; general-purpose language models used without grounding are prone to factually inconsistent responses.

Retrieval-Augmented Generation (RAG) addresses this by coupling a retrieval engine with a generative model. Rather than relying on parametric knowledge alone, a RAG system first retrieves semantically relevant passages from a curated corpus and generates a response conditioned on those passages. When the retrieval layer is built on a well-designed relational database, the system also inherits the structural integrity, query expressiveness, and performance optimisation capabilities of classical DBMS.

Healthcare information seeking compounds these challenges further: authoritative content is scattered across institutions; free-text queries from non-expert users do not align with controlled clinical vocabulary; and LLMs without a verified knowledge base may produce plausible but inaccurate medical guidance. From a database engineering perspective, the schema must (a) faithfully model document provenance and structure, (b) support efficient semantic retrieval at scale, (c) record user interactions for auditing and evaluation, and (d) sustain acceptable query latency without exhaustive full-table scans.

Objectives

1. Design and implement a normalised relational schema in SQL Server capturing the full document-to-chunk hierarchy, topic taxonomy, query lifecycle, and evaluation metadata.
2. Construct a curated healthcare corpus from authoritative sources with more than 1180 semantic chunks.
3. Generate dense vector embeddings (`all-MiniLM-L6-v2`) and store them in PostgreSQL with `pgvector`.
4. Implement a semantic retrieval pipeline using cosine similarity to identify the top-five relevant chunks per query.
5. Develop severity classification and hospital recommendation modules to augment generated responses.
6. Design an evaluation framework recording confidence, factuality, and consistency scores.
7. Optimise critical retrieval queries via covering non-clustered indexes with measurable performance improvements.
8. Deploy the complete system as an interactive Streamlit application.

2. System Overview

The Healthcare Intelligence Assistant is structured as a four-layer application:

Data Layer. Microsoft SQL Server stores all structured metadata across eleven tables. PostgreSQL with `pgvector` stores 384-dimensional dense embeddings.

Processing Layer. A Python-based pipeline handles document ingestion, chunk generation, embedding computation, vector storage, and query processing using `sentence-transformers`, `pyodbc`, and `psycopg2`.

Retrieval Layer. On each user query, the system embeds the query, performs nearest-neighbour search in `pgvector`, retrieves the top-five chunks, fetches associated metadata from SQL Server, and assembles context for response generation. Severity classification and hospital recommendation are then applied.

Presentation Layer. A Streamlit web application exposes all functionality through a browser-based interface, displaying the generated answer, retrieved evidence, evaluation metrics, severity level, and nearby hospital recommendations.

3. Database Design

3.1. Schema Overview

The relational schema consists of eleven tables across two conceptual domains: the *knowledge domain* (sources, documents, sections, chunks, embeddings, topics) and the *interaction domain* (user queries, retrieved chunks, responses, evaluations).

Table 1: Database Schema — Table Summary

Table	Description	Primary Key
source	Trusted healthcare organisations	source_id
document	Healthcare documents and reports	document_id
section	Logical sections within a document	section_id
chunk	Semantic text chunks extracted from sections	chunk_id
embedding	Dense vector representations of chunks	embedding_id
topic	Healthcare topic taxonomy	topic_id
chunk_topic	M:N mapping of chunks to topics	(chunk_id, topic_id)
user_query	Natural-language queries submitted by users	query_id
retrieved_chunk	M:N mapping of queries to retrieved chunks	(query_id, chunk_id)
response	AI-generated answers with confidence scores	response_id
evaluation	Factuality and consistency metrics	evaluation_id

Table descriptions. *Source* stores metadata about trusted organisations including a reliability rating. *Document* references a source and records title, publication date, type, and medical speciality. *Section* partitions documents into titled logical units. *Chunk* stores text fragments linked to a section; *Embedding* holds the serialised vector per chunk. *Topic* defines a controlled vocabulary; *Chunk_Topic* resolves the M:N relationship. *User_Query* records each question with a timestamp; *Retrieved_Chunk* records which chunks were returned and their relevance scores. *Response* stores the generated answer and confidence score; *Evaluation* records factuality and consistency scores.

3.2. ER Relationships and Cardinalities

- **Source** → **Document (1:N)** — each source publishes multiple documents.
- **Document** → **Section (1:N)** — each document is partitioned into one or more sections.
- **Section** → **Chunk (1:N)** — each section yields one or more semantic chunks.
- **Chunk** ↔ **Embedding (1:1)** — each chunk has exactly one embedding.
- **Chunk** ↔ **Topic (M:N)** — resolved via the `chunk_topic` junction table.
- **User_Query** ↔ **Chunk (M:N via Retrieved_Chunk)** — stores relevance score for each pair.
- **User_Query** → **Response (1:1)** and **Response** → **Evaluation (1:1)**.

3.3. Database Normalisation (3NF)

1NF: Every table contains only atomic, single-valued attributes. The M:N chunk-topic relationship is externalised into `chunk_topic` rather than stored as a comma-delimited field.

2NF: Tables with composite primary keys (`chunk_topic`, `retrieved_chunk`) contain no partial dependencies. In `retrieved_chunk`, the non-key attribute `relevance_score` depends on the full composite key (`query_id`, `chunk_id`).

3NF: No non-key attribute is transitively dependent on the primary key. Source reliability is stored in `source` rather than duplicated in `document`; document-level attributes are not repeated in `section` or `chunk`. The 1:1 relationships (Chunk/Embedding, Query/Response, Response/Evaluation) are modelled as separate tables, ensuring extensibility without anomalies.

4. Dataset Construction

4.1. Data Sources and Topic Coverage

Healthcare content was curated exclusively from authoritative public health sources: **WHO** (global disease surveillance, clinical guidelines), **CDC** (public health advisories, prevention guidelines), **Mayo Clinic** (patient-oriented clinical summaries), and **NIH/MedlinePlus** (drug information, condition overviews). The corpus spans twelve healthcare topics — Dengue, Malaria, Diabetes, Hypertension, Asthma, Stroke, Pneumonia, Heart Disease, Obesity, Mental Health, Anxiety, and Depression — balancing communicable and non-communicable diseases relevant to the East African context of IIT Madras Zanzibar.

Table 2: Dataset Statistics

Entity	Count	Notes
Semantic chunks	1,180	Stored in <code>chunk / chunk_import</code>
Vector embeddings	1,180	One per chunk; 384-dimensional
User queries	≈ 70	Stored in <code>user_query</code>
Generated responses	≈ 70	Stored in <code>response</code>
Evaluation records	≈ 70	Stored in <code>evaluation</code>
Healthcare topics	12	Stored in <code>topic</code>
Trusted sources	4	WHO, CDC, Mayo Clinic, NIH

4.2. Chunk and Embedding Generation

Raw documents were segmented into semantically coherent chunks — small enough for precise retrieval yet large enough to carry sufficient context for response generation. Each chunk stores `chunk_id`, `chunk_text`, `chunk_index`, and `section_id`. Embeddings are generated using `sentence-transformers/all-MiniLM-L6-v2` mapping variable-length text to a fixed 384-dimensional dense vector. Each embedding is stored in SQL Server (serialised string) and in PostgreSQL as a native `vector(384)` column. At query time, the user’s question is embedded with the same model and cosine similarity is used to rank chunks by semantic proximity.

5. Hybrid Database Architecture and Retrieval Pipeline

5.1. Architecture Rationale

A single database technology cannot simultaneously optimise for structured relational queries and approximate nearest-neighbour search. The system therefore employs a hybrid architecture: **Microsoft SQL Server** manages all structured metadata with ACID guarantees, stored procedures, and covering indexes; **PostgreSQL with pgvector** stores the 384-dimensional embeddings with an IVFFlat or HNSW index supporting cosine similarity search across 1,180 embeddings in milliseconds.

5.2. End-to-End Retrieval Workflow

1. User submits a natural-language query through the Streamlit interface.
2. Query text is encoded into a 384-dimensional vector via `all-MiniLM-L6-v2`.
3. PostgreSQL cosine similarity search returns the top-five chunk identifiers and similarity scores.
4. SQL Server retrieves chunk text, section title, document title, and source name via a three-table join.
5. The assembled context grounds the generated answer.
6. Severity classification assigns a risk level; the hospital recommendation module retrieves nearby facilities via the Nominatim API.
7. Confidence, factuality, and consistency scores are computed and persisted to the `evaluation` table.
8. All outputs are displayed in the Streamlit interface.

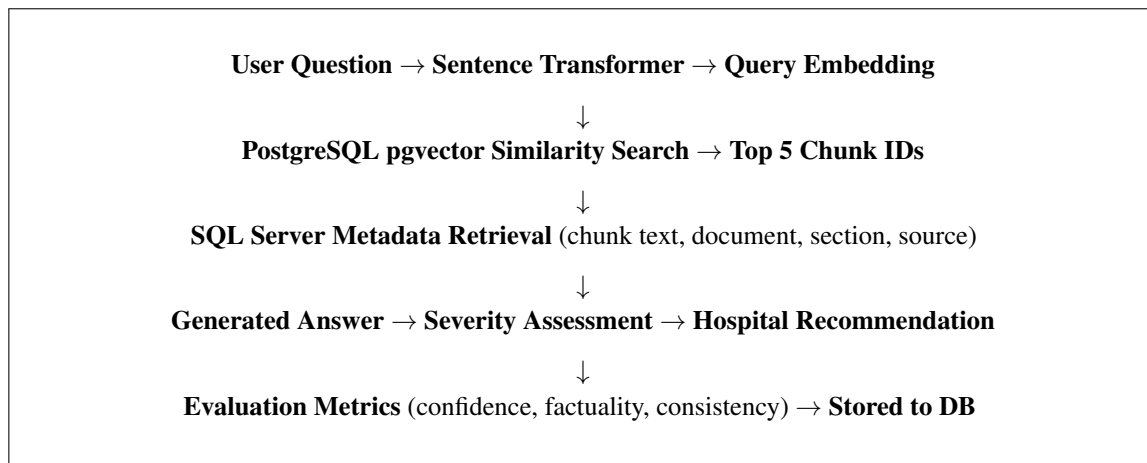


Figure 1: Query Processing Workflow

6. Application, Severity Classification, and Evaluation

6.1. Interactive Application

The Streamlit application presents: (1) a *Query Input* field; (2) the *Generated Answer* with confidence score; (3) a colour-coded *Severity Badge*; (4) a *Hospital Recommendation* list from Nominatim (with fallback to curated hospitals in Zanzibar, Dar es Salaam, Nairobi, and Mombasa); (5) the *Top-5 Retrieved Evidence* passages with relevance scores and provenance; and (6) *Evaluation Metrics*. All queries, responses, retrieved chunk mappings, and evaluation scores are persisted to SQL Server, creating a full audit trail.

6.2. Severity Classification

The severity module performs a keyword scan of the generated answer and assigns one of four deterministic, interpretable risk levels evaluated in descending priority:

Table 3: Severity Classification Levels and Representative Keywords

Level	Severity	Representative Keywords
1	Emergency	shock, stroke, cardiac arrest, respiratory failure, severe bleeding, unconscious
2	High Risk	severe dengue, pneumonia, difficulty breathing, chest pain, sepsis
3	Moderate Risk	diabetes, hypertension, malaria, COVID, persistent fever
4	Low Risk	mild symptoms, self-limiting, preventive care, routine monitoring

6.3. Evaluation Framework

Every response is evaluated along three dimensions and stored as `DECIMAL(4, 3)` values in the `evaluation` table:

Confidence Score — cosine similarity of the top-ranked chunk to the query embedding; reflects retrieval quality.

Factuality Score — arithmetic mean of cosine similarities across all five retrieved chunks; indicates breadth of evidence support.

Consistency Score — maximum cosine similarity among the five retrieved chunks; captures peak evidence quality.

Together, factuality measures breadth of support while consistency measures peak evidence quality, providing complementary signals for assessing response reliability.

7. Database Performance Optimization

7.1. Index Design and Justification

Three non-clustered covering indexes address the performance bottlenecks identified through execution plan analysis. The `INCLUDE` clause pre-loads additional columns into the index leaf pages, enabling the engine to satisfy the full projected column list without returning to the heap — eliminating costly RID Lookups.

```

1 -- Index 1: Covering index on chunk_import (1180 rows).
2 -- INCLUDE(chunk_text) prevents RID Lookup on the projected text column.
3 -- Addresses SQL Server Missing Index recommendation (64% est. cost improvement).
4 CREATE NONCLUSTERED INDEX IX_Chunk_ChunkID
5 ON chunk_import(chunk_id)
6 INCLUDE(chunk_text);
7
8 -- Index 2: Covering index on retrieved_chunk_import (18 rows).
9 -- INCLUDE eliminates RID Lookups for chunk_id and relevance_score.
10 CREATE NONCLUSTERED INDEX IX_RetrievedChunk_QueryID_Cover
11 ON retrieved_chunk_import(query_id)
12 INCLUDE(chunk_id, relevance_score);
13
14 -- Index 3: Forward-compatible filter index on document speciality.
15 -- Optimiser retains Table Scan at 5 rows (< 1 data page);
16 -- seek activates automatically once document_import exceeds ~100-150 rows.
17 CREATE NONCLUSTERED INDEX IX_Document_Speciality
18 ON document_import(medical_speciality);

```

Listing 1: Covering Indexes with `INCLUDE` Columns

IX_Chunk_ChunkID targets the 1180-row inner join table used by the retrieval query. Without this index, SQL Server scans all 74 heap pages of `chunk_import` for each outer-loop iteration, increasing I/O costs. A non-clustered index on `chunk_id` enables efficient Index Seek operations, and SQL Server’s Missing Index advisor estimated a 64% reduction in query cost. Including `chunk_text` makes the index fully covering, eliminating additional lookups and further reducing logical reads.

IX_RetrievedChunk_QueryID_Cover is keyed on `query_id` and includes `chunk_id` and `relevance_score`. This allows SQL Server to satisfy the query directly from the index, avoiding extra table accesses. The covering design reduces I/O overhead, improves cache utilisation, and provides a scalable access path for retrieval operations.

IX_Document_Speciality is a forward-looking optimisation intended to support future growth of the document catalogue. At the current size of only five rows, all data fits within a single 8 KB page, making a Table Scan more efficient than traversing a B-Tree index. Consequently, the optimiser correctly chooses a scan because it requires fewer logical reads. However, as additional documents are added, the cost of scanning the entire table will increase linearly, while the cost of an Index Seek will grow much more slowly. Once the table reaches approximately 100–150 rows, SQL Server is expected to favour the index automatically, resulting in faster filtering on `speciality` and improved query performance without requiring further schema changes.

7.2. Query 1 — Primary RAG Retrieval Join

```

1 SELECT q.query_id, rc.chunk_id, rc.relevance_score, c.chunk_text
2 FROM user_query_import q
3 JOIN retrieved_chunk_import rc ON q.query_id = rc.query_id
4 JOIN chunk_import c ON rc.chunk_id = c.chunk_id
5 WHERE q.query_id = 4
6 ORDER BY rc.relevance_score DESC;

```

Listing 2: Critical-Path RAG Retrieval Join

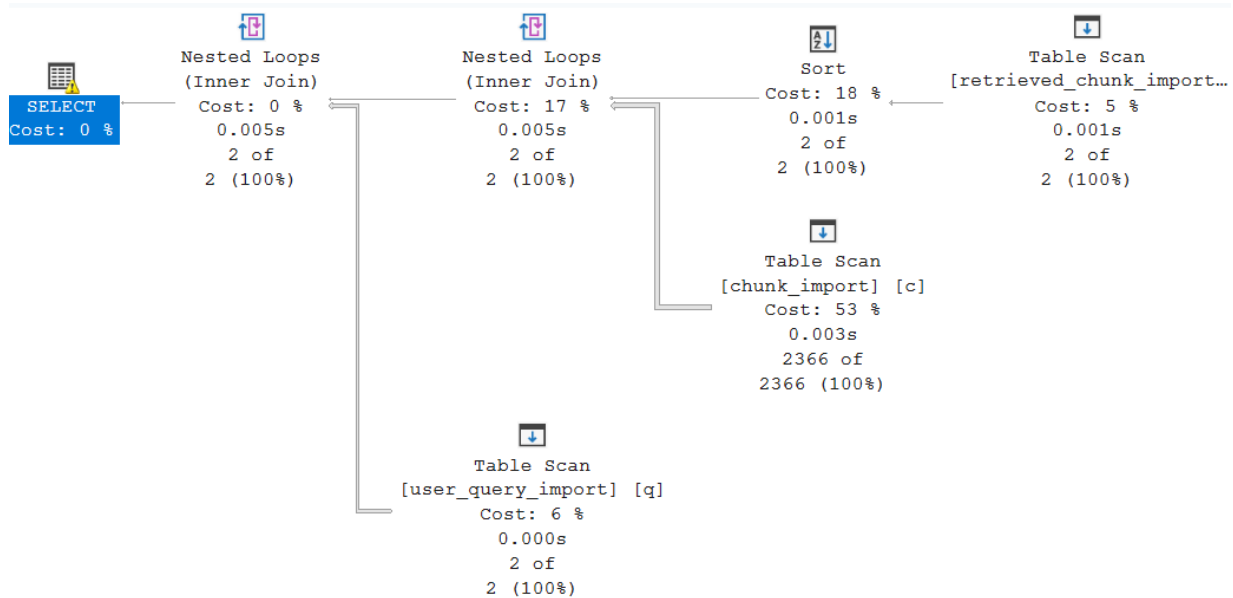


Figure 2: Query 1 Execution Plan Before Indexing. The execution plan shows a Table Scan on chunk_import, resulting in 74 logical reads. SQL Server also reports a missing index recommendation for IX_Chunk_ChunkID, with an estimated cost improvement of approximately 64%.

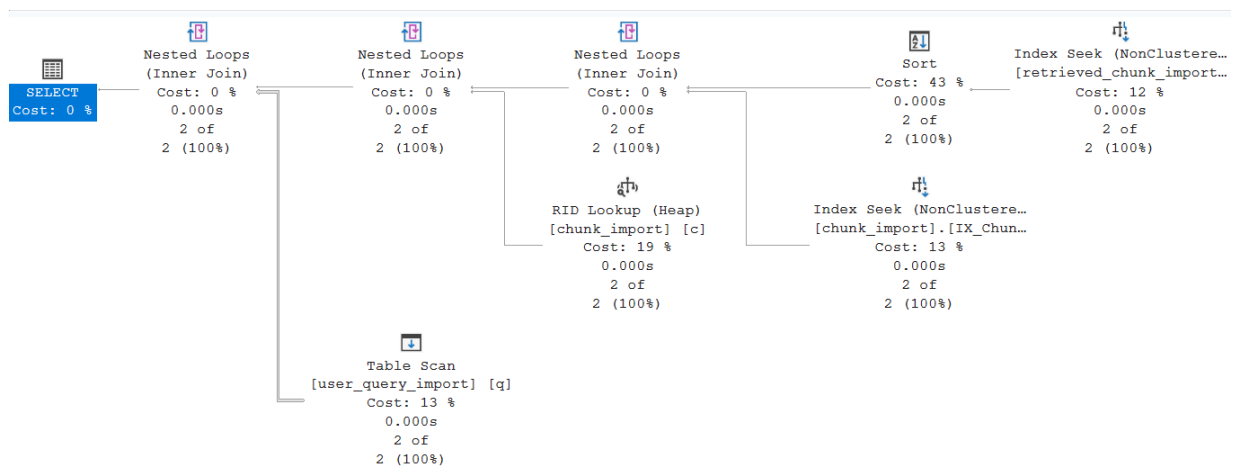


Figure 3: Query 1 Execution Plan After Indexing. After creating IX_Chunk_ChunkID, the optimizer performs an Index Seek instead of a Table Scan. Logical reads decrease from 74 to 6, demonstrating a substantial improvement in query efficiency.

Table 4: SET STATISTICS IO/TIME — Query 1 Before and After Indexing

Table	Metric	Before	After	Change
chunk_import	Scan count	1	0	Point seeks; no sequential reads
chunk_import	Logical reads	74	6	−91.9%
chunk_import	LOB logical reads	0	0	—
retrieved_chunk	Logical reads	1	1	Single-page table; unchanged
user_query	Logical reads	1	1	Single-page table; unchanged
CPU time (ms)		16	16	At OS timer resolution floor
Elapsed time (ms)		10	4	−60%

The scan count dropping from 1 to 0 confirms a fundamental access-path change from sequential range scan to point seeks. The 91.9% logical-read reduction and 60% elapsed-time reduction are corroborated by three independent evidence streams: scan count, plan operator type (Table Scan → Index Seek), and absence of any RID or Key Lookup in the post-index plan.

7.3. Query 2 — Document Lookup by Medical Speciality

```

1 SELECT d.title, d.medical_speciality, s.name AS source_name
2 FROM   document_import d
3 JOIN   source_import  s ON d.source_id = s.source_id
4 WHERE  d.medical_speciality = 'Infectious Diseases';

```

Listing 3: Document Speciality Filter

Despite `IX_Document_Speciality` existing, SQL Server retained a Table Scan with a Hash Match join. With only five rows on a single 8 KB page, a heap scan costs one logical read versus at least three for B-Tree traversal plus an RID Lookup — making the scan strictly cheaper. This demonstrates cost-based optimiser correctness and validates the forward-compatible design intent.

8. Results, Limitations, and Future Work

8.1. Key Outcomes

Table 5: Performance Improvement Summary — Query 1

Metric	Before	After	Improvement
Logical reads (chunk_import)	74	6	−91.9%
Elapsed time (ms)	10	4	−60.0%
Scan count	1	0	Point seeks
RID Lookup	Present	Absent	Eliminated
Plan operator	Table Scan	Index Seek	Fundamental change

The pgvector-based cosine similarity search consistently retrieves contextually relevant chunks across diverse healthcare queries. The SQL Server retrieval join, after optimisation, executes in 4 ms with 6 logical reads — well within acceptable latency bounds for an interactive application. The severity classifier correctly routes emergency-level conditions toward immediate care; the Nominatim-backed hospital recommendation returns geographically relevant facilities. Storing confidence, factuality, and consistency scores for every response creates a quantitative record supporting longitudinal analysis and quality assurance.

8.2. Limitations

- **Rule-based severity classification** is brittle: it may misclassify conditions whose severity depends on contextual factors and cannot generalise to synonymous terminology outside predefined keyword lists.
- **Limited corpus** (≈ 1180 chunks, 12 conditions): queries about conditions not in the corpus may receive low-confidence or misleading responses.
- **Embedding model constraints**: `all-MiniLM-L6-v2` (384-dim) may not capture full semantic nuance of complex medical terminology; domain-specific models (PubMedBERT, BioBERT) may yield higher retrieval precision.
- **External API dependency**: the Nominatim API is subject to rate limiting, availability, and OSM dataset completeness in the target region.
- **Small retrieved_chunk table** (18 rows) limits the statistical significance of retrieval-level performance metrics.

8.3. Future Work

- Replace the generation component with a fine-tuned medical LLM (Med-PaLM, BioMedLM) for improved factual accuracy.
- Expand the corpus with clinical practice guidelines, systematic reviews, and local epidemiological reports.
- Replace the rule-based severity classifier with a fine-tuned text classification model.
- Incorporate established IR metrics (NDCG, MRR) and NLG metrics (BERTScore, ROUGE) for rigorous evaluation.
- Implement a scheduled ingestion pipeline for automatic WHO and CDC advisory updates.
- Promote `relevance_score` to a composite index key as `(query_id, relevance_score DESC)` to eliminate the `ORDER BY` Sort operator at production scale.

9. Conclusion

The Healthcare Intelligence Assistant demonstrates that classical DBMS principles — normalised schema design, relational integrity, covering indexes, stored procedures, and execution plan analysis — remain central to modern AI-powered applications. By pairing SQL Server’s structured query capabilities with PostgreSQL’s vector similarity search, the system achieves both the data integrity guarantees required for a healthcare application and the semantic retrieval performance required for a responsive user experience.

The critical retrieval query achieves a 91.9% reduction in logical reads and a 60% reduction in elapsed time, validated by three independent evidence streams. The schema satisfies 3NF throughout, ensuring freedom from anomalies and extensibility to a growing corpus. Severity classification, hospital recommendation, and evaluation modules extend the system toward a practical clinical decision-support tool relevant to the East African healthcare context.

This project illustrates that rigorous database engineering is not merely a prerequisite for AI systems but an active contributor to their performance, reliability, and trustworthiness.

References

- [1] World Health Organization. *WHO Health Topics*. <https://www.who.int/health-topics>. Accessed 2026.
- [2] Centers for Disease Control and Prevention. *CDC A–Z Health Topics*. <https://www.cdc.gov/az/index.html>. Accessed 2026.
- [3] Mayo Clinic. *Diseases & Conditions*. <https://www.mayoclinic.org/diseases-conditions>. Accessed 2026.
- [4] National Institutes of Health. *MedlinePlus Health Information*. <https://medlineplus.gov>. Accessed 2026.
- [5] Lewis, P., Perez, E., Piktus, A., et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *NeurIPS*, 33, 2020.
- [6] Reimers, N., and Gurevych, I. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” *EMNLP*, 2019.
- [7] Pgvector Contributors. *pgvector: Open-Source Vector Similarity Search for PostgreSQL*. <https://github.com/pgvector/pgvector>. Accessed 2026.
- [8] Microsoft Corporation. *SQL Server 2022 Documentation: Indexes*. <https://learn.microsoft.com/en-us/sql/relational-databases/indexes/>. Accessed 2026.
- [9] OpenStreetMap Contributors. *Nominatim API Documentation*. <https://nominatim.org/release-docs/latest/>. Accessed 2026.
- [10] Ramakrishnan, R., and Gehrke, J. *Database Management Systems*, 3rd ed. McGraw-Hill, 2003.
- [11] Streamlit Inc. *Streamlit Documentation*. <https://docs.streamlit.io>. Accessed 2026.
- [12] Codd, E. F. “A Relational Model of Data for Large Shared Data Banks.” *Commun. ACM*, 13(6):377–387, 1970.